Relatório — Classificação de complexidade textual e geração de textos por mutação

Prova de seleção 024-2026 · CAEd

1. Resumo

Construí um pipeline completo: limpeza do dataset → instâncias curtas → features (incluindo canonicidade silábica) → comparação de 10 modelos → três operadores de mutação → medição do efeito das mutações na complexidade prevista → aplicação web. Principais resultados (teste final em material não visto, 4 classes):

Modelo	**F1 macro (CV por blocos)**	**F1 macro (teste)**	**Acurácia (teste)**
Baseline (classe majoritária)	0,142	0,144	0,406
HistGradientBoosting · 69 features	0,610	0,586	0,609
TF-IDF de caracteres (2–5) · Regressão Logística	0,753	0,755	0,793

Três conclusões que considero mais importantes: (i) a forma de dividir treino/teste muda tudo (o mesmo modelo vai de 0,75 para 0,87 de F1 numa divisão aleatória, por vazamento); (ii) a canonicidade ajuda pouco como feature isolada; (iii) as mutações quase não alteram a complexidade prevista (variação média ≤ 0,03 numa escala de 0 a 3), o que serve bem ao objetivo de criar textos novos de nível equivalente.

Código, modelos, dados processados e instruções de execução: repositório `https://github.com/SEU_USUARIO/caed-complexidade` e README.md.

2. Dados e pré-processamento

O dataset tem 2.076 arquivos (297 EF-I, 325 EF-II, 628 EM, 826 ES), cada um com 300+ palavras.

Limpeza. Inspecionando os arquivos, ~30% das linhas de EM e ES são títulos, itens de índice ou caminhos de navegação (`Cálculo/Geometria tridimensional`, `Português/Imprimir`), contra ~0% no EF-I. Isso é um atalho perigoso: o modelo poderia aprender "formatação suja = nível alto". Removi linhas curtas sem pontuação final, linhas com muitos símbolos/números e caminhos com `/` (`caed/data.py::clean_lines`).

Instâncias. Cada arquivo foi quebrado em trechos de ~55 palavras (mínimo 30, máximo 130), agrupando sentenças consecutivas do mesmo parágrafo, porque o enunciado fala em "pequeno texto". Resultado: 11.620 instâncias (EF-I 1.594; EF-II 1.903; EM 3.472; ES 4.651). As classes são desbalanceadas, então usei `class_weight="balanced"` e F1 macro como métrica principal.

Observação sobre o dataset. Parte dos textos do Ensino Superior está em português europeu (oxigénio, objectivo, protecção). Isso é um segundo atalho possível (grafia ≈ nível). Não o removi; apenas registro como limitação (seção 8).

3. Canonicidade de palavras (`caed/syllables.py`)

Palavra canônica = todas as sílabas são CV. Implementei um silabificador por regras (princípio do onset máximo) e a canonicidade é derivada das sílabas:

1. Dígrafos `ch lh nh rr ss` contam como uma consoante; `qu`/`gu` antes de e/i também (o u é mudo); antes de a/o (*quando*) o u é semivogal, então a sílaba não é CV.
2. Ditongos e hiatos geram núcleo VV ou sílaba só-vogal → não canônica (*pai, mãe, asa, alheio*).
3. Coda (m/n nasal, s, r, l, z finais) → CVC → não canônica (*cansado, lápis, abajur*).
4. Encontros que podem iniciar sílaba (`bl br cl cr dr fl fr gl gr pl pr tr vr`) ficam juntos (*pro-ble-ma*); os demais se dividem (*pas-ta*).

Decisão consciente: o `h` é tratado como consoante (ortográfico), então *há/hoje* contam como CV. Os exemplos do enunciado são todos classificados corretamente (`tests/test_syllables.py`: cabeça, canela, janela, luta, sopa → canônicas; abajur, lápis, cansado, asa, alheio → não canônicas).

Features derivadas: proporção de palavras canônicas (tokens e tipos), sílabas não-CV por palavra e distribuição das formas silábicas (CV, V, CVC, CCV, CVV…).

4. Modelo de predição

4.1 Features (69)

Superficiais (tamanho de frase, tamanho de palavra, sílabas por palavra, Flesch adaptado, pontuação), lexicais (TTR, MATTR), silábicas/canonicidade e sintáticas via spaCy `pt_core_news_sm` (proporção de classes gramaticais, subordinação, voz passiva, distância de dependência, profundidade da árvore, modo/tempo verbal).

4.2 Protocolo de avaliação (parte mais importante)

Arquivos vizinhos (por número de ordem no nível) vêm do mesmo material didático, e trechos do mesmo livro compartilham vocabulário e estilo. Uma divisão aleatória de instâncias deixaria trechos do mesmo material em treino e teste. Por isso:

- cada nível foi dividido em 5 blocos contíguos de arquivos;
- blocos 0–3 = desenvolvimento, com validação cruzada de 4 dobras (cada dobra = um bloco inteiro);
- bloco 4 = teste final, usado só depois de escolher o modelo;
- para medir o vazamento, rodei também uma divisão aleatória (propositalmente errada).

Verificação da suposição de blocos. Essa estratégia assume que a ordem numérica dos arquivos reflete proximidade de material. Para checar, medi a similaridade TF-IDF (cosseno) entre arquivos consecutivos e entre pares distantes de cada nível. Arquivos vizinhos são mais parecidos que arquivos distantes (por exemplo, no Ensino Médio, ≈0,19 contra ≈0,05), e a similaridade decai com a distância, o que indica que a ordem numérica de fato reflete proximidade de material. Porém, algumas fronteiras de bloco cortam materiais: o cosseno entre arquivos em lados opostos de uma fronteira chega a ≈0,26–0,38 em alguns casos, inclusive na fronteira entre desenvolvimento e teste do Ensino Superior. Há, portanto, um **vazamento residual pequeno, que não corrigi** por falta de tempo. Ele pode inflar um pouco o resultado do teste, principalmente nos modelos TF-IDF, e eu não testei se isso altera o ranking dos modelos. O ajuste natural seria descartar alguns arquivos de cada lado das fronteiras e retreinar.

4.3 Resultados

Modelo	CV F1	Teste F1	Teste acc	MAE ordinal
Baseline (majoritária)	0,142	0,144	0,406	1,03
LR · só tamanho/sílabas	0,400	0,330	0,338	1,02
LR · features (sem canonicidade)	0,563	0,496	0,508	0,64
LR · features	0,576	0,518	0,536	0,60
Random Forest · features	0,568	0,515	0,561	0,54
HistGB · features (sem canonicidade)	0,603	0,561	0,577	0,54
HistGB · features	0,610	0,586	0,609	0,49
TF-IDF palavras · LR	0,744	0,728	0,781	0,25
TF-IDF caracteres · LR	**0,753**	**0,755**	**0,793**	**0,24**
Híbrido: features + TF-IDF palavras · LR	0,705	0,678	0,716	0,34

(CV = média de 4 dobras, desvio ≈ 0,02; MAE = erro médio em níveis.)

Vazamento. Com divisão aleatória, F1 sobe para 0,87 (TF-IDF palavras e caracteres), 0,81 (híbrido) e 0,66 (HistGB features). O ganho do TF-IDF é muito maior que o das features de estrutura: ele memoriza vocabulário de material.

Escolha. O melhor na validação por blocos foi TF-IDF de caracteres + LR (0,753), embora estatisticamente empatado com TF-IDF de palavras (0,744), dado o desvio de ≈0,02 entre dobras. Usei o de caracteres como modelo principal no app. Mantive também o HistGB sobre features como modelo **interpretável**, exibido junto no app.

Confusões. No teste, o EF-II é a classe mais difícil: só 195 de 352 trechos (55%) foram acertados, a maioria confundidos com EF-I. Erros entre níveis vizinhos dominam (97% das previsões erram no máximo 1 nível).

4.4 O que a canonicidade agrega

- Ablação (CV por blocos): LR 0,563 → 0,576 (+0,013); HistGB 0,603 → 0,610 (+0,007). No teste: +0,022 e +0,025.
- A proporção média de palavras canônicas é quase igual entre níveis (EF-I 0,230; EF-II 0,220; EM 0,229; ES 0,221). A diferença está em tipos únicos (0,201 no EF-I contra 0,179–0,194) e em sílabas não-CV por palavra (1,05 no EF-I contra 1,10–1,14).
- Conclusão honesta: a feature carrega algum sinal, mas **pequeno** neste dataset; o grosso vem de tamanho de frase/palavra e vocabulário. Sinais mais úteis na permutação (HistGB): tempo verbal passado, aspas por frase, forma silábica CVV, TTR, parênteses por frase.

5. Operadores de mutação (`caed/mutations.py`)

Todos usam heurísticas sobre a árvore de dependências do spaCy; nenhum usa redes generativas.

Troca de palavras. Substitui substantivos, adjetivos, verbos e advérbios por sinônimos do OpenWN-PT (via OMW). Antônimos (só adjetivos e advérbios) vêm da relação de antonímia do WordNet inglês, projetada nos mesmos synsets. Como o WordNet-PT é ruidoso e não tem desambiguação, três filtros de qualidade:

1. sobreposição de sentidos (Jaccard dos synsets);
2. similaridade distribucional (vetores PPMI+SVD treinados no próprio corpus, sem rótulos, cosseno ≥ 0,30);
3. encaixe no contexto: o candidato precisa ser tão similar às palavras vizinhas quanto a palavra original.

A flexão é refeita: verbos pelo "banco de formas" extraído do corpus (lema+traços → forma), substantivos exigem o mesmo gênero atestado, adjetivos por regras de gênero/número. Há modo de direção (mais simples / mais difícil), que escolhe o candidato de menor ou maior "dificuldade lexical" (tamanho, sílabas, canonicidade, frequência).

Troca de gênero. Lista de ~100 pares (cantor/cantora, professor/professora…) + invariáveis (estudante, artista) + pronomes ele/ela. A concordância é propagada pela árvore: artigos, contrações (do/da, pelo/pela), adjetivos, particípios, predicativos e coordenados (*A cantora era famosa e muito querida pelos vizinhos → O cantor era famoso e muito querido pelas vizinhas*). Regras de segurança: não troca frases com nome de pessoa desconhecido (evita "Josefa Carreras"), frases só com pronome (antecedente em outra frase) nem predicativos nominais com só um lado trocável (*Um tenor é um cantor*).

Paráfrase. Uma transformação por frase, escolhida entre as aplicáveis:

- passiva → ativa (*O texto foi lido pelo aluno → O aluno leu o texto*), com conjugação via banco de formas + regras (e verbos irregulares tabelados);
- ativa → passiva (inverso, com particípio e contração pelo/pela);
- deslocamento de adjunto (*Há duas semanas atrás estava chovendo → Estava chovendo há duas semanas atrás*; *Ontem, ele chegou ↔ Ele chegou ontem*);
- aposto (*O cantor Roberto Carlos fez aniversário ↔ Roberto Carlos, o cantor, fez aniversário*);
- divisão de oração coordenada e junção de frases curtas.

Os exemplos do enunciado são casos de teste automatizados (`tests/test_mutations.py`).

6. Efeito das mutações na complexidade

Método: 240 trechos do bloco de teste (60 por nível); os dois modelos desta seção foram treinados só nos blocos 0–3. Mede-se Δ do escore contínuo E[nível] = Σ p_k·k (0–3). Intensidade 0,3 (probabilidade de cada candidato ser mutado). Escore médio por nível (TF-IDF): 0,28 / 0,80 / 1,84 / 2,60, ou seja, um nível equivale a ~0,5–1,0 de escore.

Mutação	Textos alterados	Alterações / 100 palavras	Δ médio (TF-IDF chars)	Δ médio (HistGB)	Troca de classe (TF-IDF / HistGB)
Palavras (sinônimos + antônimos)	83%	3,2	+0,010	−0,005	2,9% / 5,4%
Palavras (só sinônimos)	80%	3,1	+0,007	0,000	2,1% / 6,2%
Palavras → mais simples	54%	1,3	−0,003	**−0,022**	2,1% / 2,9%

Mutação	Textos alterados	Alterações / 100 palavras	Δ médio (TF-IDF chars)	Δ médio (HistGB)	Troca de classe (TF-IDF / HistGB)
Palavras → mais difícil	67%	2,0	+0,015	**+0,026**	2,1% / 7,1%
Troca de gênero	6%	0,2	+0,001	−0,001	0% / 0%
Paráfrase	16%	0,3	+0,003	−0,002	0,4% / 0,8%
As três juntas	89%	3,9	+0,014	+0,002	5,8% / 5,8%

Varredura de intensidade (as três juntas), Δ médio TF-IDF / HistGB: 0,1 → +0,006 / −0,003; 0,3 → +0,013 / −0,001; 0,6 → +0,026 / +0,006; 1,0 → +0,032 / +0,026.

Leitura:

- As mutações **preservam o nível previsto em ≥ 94% dos casos**, mesmo com quase todo texto sendo alterado. Para o caso de uso do CAEd (gerar texto novo de dificuldade equivalente) isso é desejável.
- A variação cresce com a intensidade, mas pouco (≈ 0,03 de escore no máximo, uma fração pequena de um nível).
- O modo de direção funciona no modelo interpretável (−0,022 contra +0,026), mas o efeito é pequeno. O TF-IDF de caracteres reage menos, coerente com ele medir mais o vocabulário de material do que a dificuldade lexical.
- **Cobertura da troca de gênero é baixa** (6% dos textos): textos didáticos têm poucos substantivos animados com par na lista. A paráfrase altera 16% dos textos a 0,3 (regras específicas; só altera quando o padrão aparece).
- Mudar a estrutura (paráfrase) mexe menos no escore que mudar o léxico, porque as features estruturais do modelo são médias sobre o trecho.

Controle positivo. "Quase não muda" poderia significar apenas que o modelo não enxerga as mudanças. O modo de direção no HistGB serve de controle positivo: trocar por palavras mais simples (−0,022) ou mais difíceis (+0,026) move o escore no sentido esperado, então o modelo consegue se mover; as demais mutações é que alteram pouco o que ele mede.

Nota sobre os números do app. Os modelos desta seção foram treinados só nos blocos 0–3. Os modelos usados no app foram reajustados com todos os dados, então os Δ mostrados na interface podem diferir levemente dos desta tabela.

7. Demonstração com os exemplos do enunciado

Rodei o app com intensidade 1 e os três operadores ligados.

Texto original	Texto mutado	Operadores aplicados	Δ complexidade
A cantora era famosa	O cantor era célebre	gênero (A→O, cantora→cantor, famosa→famoso); sinônimo (famoso→célebre)	+0,057 (Ensino Médio → Ensino Médio)

Texto original	Texto mutado	Operadores aplicados	Δ complexidade
Há duas semanas atrás, o cantor Roberto Carlos fez aniversário.	PREENCHER	PREENCHER	PREENCHER
O cantor Roberto Carlos fez aniversário.	PREENCHER	PREENCHER	PREENCHER

A paráfrase é um operador sorteado por frase e só atua quando o padrão aparece, então a semente influencia o resultado; nos casos acima usei PREENCHER (semente ou "automática").

8. Limitações e problemas encontrados

- **Atalhos no dataset:** grafia europeia e vocabulário de material podem inflar o TF-IDF. Os blocos reduzem, mas não garantem separação por material.
- **Vazamento residual entre blocos (seção 4.2):** algumas fronteiras cortam materiais, então o teste pode estar levemente otimista para os modelos TF-IDF. Não corrigido.
- **EF-II é fraco** (recall 55%): textos de fronteira entre níveis.
- **Troca de palavras:** sem desambiguação real de sentido. Numa revisão qualitativa informal de 43 trocas sinônimas (não é um estudo formal), cerca de 1 em cada 5 foi inadequada (*eleição → alternativa, ligada → chamada, existem → são*). Os filtros distribucional e de contexto ajudaram (antes: *anos → sonos, corpo → morto*), mas não eliminam o problema. Antônimos mudam o sentido por construção. Verbos podem trocar regência (*mostrar → apontar*).
- **Troca de gênero:** cobertura limitada pela lista de pares; predicativos secundários mal etiquetados pelo tagger pequeno às vezes não concordam (*A mulher saiu sozinha → O homem saiu sozinha*); pronomes não são rastreados entre frases.
- **Paráfrase:** baseada na análise do spaCy `sm`; erros de parse geram saídas ruins. Aplica poucas vezes.
- Não fiz avaliação humana de gramaticalidade/preservação de sentido; as métricas da seção 6 medem só o efeito no modelo.
- Os modelos de complexidade foram treinados com trechos curtos (~55 palavras); textos muito diferentes disso podem ter previsões menos confiáveis.

9. Ideias para continuar

Desambiguação de sentido com embeddings contextuais; geração de paráfrases por modelo de linguagem, controlada por alvo de complexidade; classificação ordinal (regressão ordinal em vez de 4 classes); coletar o identificador de material para fazer divisão por material de verdade (ou descartar arquivos nas fronteiras dos blocos); modelo de frequência lexical externo (corpus geral) para medir familiaridade das palavras.

10. Ferramentas

Python 3.13 (testado; o README indica 3.11+), spaCy (`pt_core_news_sm`), scikit-learn, NLTK (leitura do WordNet), OpenWN-PT via OMW, Flask, matplotlib. Código, dados processados, modelos e instruções de execução em README.md e no repositório.